

Memoria M3

Optimización de carteras · Roboadvisor TFM

Construcción de tres carteras óptimas long-only UCITS — una por perfil de riesgo (**conservador**, **moderado**, **agresivo**) — combinando Mínima Varianza, Máximo Sharpe y Hierarchical Risk Parity, con caps de volatilidad gestionados mediante *cash blending*.

Resultado

Tres carteras seleccionadas — todas **PASS** en validación UCITS y vol cap. Sharpe ex-ante: **-0.127** (conservador, equal-weight), **0.449** (moderado, max-sharpe), **0.899** (agresivo, max-sharpe). Suite de 16 tests automatizados. Outputs (weights.parquet + portfolios_summary.json) listos para M4.



Índice

1. Contexto y objetivo
2. Inputs heredados de M2
3. Arquitectura del módulo m3_portfolio
4. Restricciones del problema
5. Estrategias de optimización por perfil
6. Manejo del vol cap: blending con cash
7. Resultados: 9 carteras candidatas
8. Selección final por perfil
9. Validación UCITS y de volatilidad
10. Tests automatizados
11. Outputs y handover a M4
12. Notas metodológicas y limitaciones
13. Nota metodológica: cartera ex-ante vs cartera OOS-clean
- A. Apéndice — Inventario de ficheros

1 Contexto y objetivo

El **M3** es la fase de construcción de carteras óptimas del roboadvisor. Recibe del M2 un universo de 9 ETFs UCITS (3 por perfil) junto con sus retornos esperados y matriz de covarianza, y devuelve **tres carteras óptimas** — una por perfil — listas para evaluación en el M4 con backtesting walk-forward.

El módulo se diseña como **paquete Python instalable** (`m3_portfolio`), independiente del notebook que lo orquesta. Esta separación permite reutilizarlo en M4 (rebalanceo periódico), testarlo con `pytest` y aplicar el principio de separación de responsabilidades.

Audiencia

Tribunal de Data Science. La documentación se centra en arquitectura de software y resultados; los desarrollos matemáticos (frontera eficiente, clustering jerárquico) se referencian pero no se reproducen.

2 Inputs heredados de M2

El M3 **no recalcula** ni μ ni Σ : confía en los artefactos del M2. Esto evita inconsistencias entre fases y respeta el principio de *single source of truth*.

Fichero	Forma	Contenido
ETFs/returns.parquet	(3 941, 9)	Retornos diarios 2010-09 → 2026-05
ETFs/mu.pkl	Series[9]	μ anualizado (media $\times$ 252)
ETFs/cov_ledoit_wolf.pkl	DataFrame[9 $\times$ 9]	Σ Ledoit-Wolf ($\alpha \approx 0.0073$, PD)

Universo por perfil:

Perfil	Tickers	Descripción
Conservador	IEAG.AS · EUNH.DE · IBCI.AS	Renta fija EUR
Moderado	INFR.AS · IHYG.L · EXSA.DE	Mixto global
Agresivo	CSPX.L · EQQQ.DE · EXI2.DE	Renta variable global

3 Arquitectura del módulo `m3_portfolio`

El paquete sigue un patrón **Strategy + Protocol estructural**:

- **PortfolioOptimizer** (PEP 544 Protocol) define el contrato mínimo: `optimize(mu, cov, tickers, max_volatility, profile, risk_free_rate) → Portfolio`.
- Cada **optimizador concreto** (Min-Variance, Max-Sharpe, HRP, Equal-Weight) implementa el método. La matemática vive en `PyPortfolioOpt`; el paquete actúa como **adaptador**.
- Todos delegan en `_build.build_portfolio()`, que proyecta al simplex UCITS, aplica el vol cap (cash) y calcula las métricas. Garantiza que **ningún Portfolio sale del módulo violando restricciones**.

Estructura de ficheros:

```
roboadvisor/m3_portfolio/
■■■■ __init__.py · API pública
■■■■ portfolio.py · @dataclass(frozen) Portfolio
■■■■ base.py · Protocol PortfolioOptimizer
■■■■ constraints.py · UCITS + vol cap + HHI
■■■■ validators.py · validación post-hoc
```

```

■■■ _build.py · ensamblador compartido
■■■ min_variance.py · MinVarianceOptimizer
■■■ max_sharpe.py · MaxSharpeOptimizer (con fallback)
■■■ hrp.py · HRP0Optimizer
■■■ equal_weight.py · EqualWeightOptimizer (baseline)

```

Modelo de datos

Portfolio es **frozen** (inmutable). Contiene: profile, optimizer_name, weights, cash_weight, expected_return, expected_volatility, expected_sharpe, herfindahl_index, n_effective_assets, constraints_satisfied, validation_report. La inmutabilidad evita modificaciones accidentales y facilita el paso por valor a M4.

4 Restricciones del problema

UCITS (estructurales):

- **Long-only:** $w_i \geq 0$.
- **Sin apalancamiento:** $\sum w_i = 1$.

Se aplican vía proyección al simplex en `apply_ucits_constraints()`: clip a 0 los pesos negativos y renormalización.

Cap de volatilidad por perfil:

Perfil	max_volatility
Conservador	8 %
Moderado	15 %
Agresivo	25 %

Restricción a nivel de cartera (no por activo). Aplicada *ex post* mediante blending con cash (\$6).

5 Estrategias de optimización por perfil

Perfil	Principal	Alternativa	Baseline
Conservador	Mínima varianza	Máximo Sharpe	Equal-weight
Moderado	Máximo Sharpe	HRP	Equal-weight
Agresivo	HRP	Máximo Sharpe	Equal-weight

Justificación:

- **Conservador** → **Min-Variance**. Maximiza robustez sin usar μ (ruidoso en RF EUR).
- **Moderado** → **Max-Sharpe**. Punto de tangencia clásico de Markowitz.
- **Agresivo** → **HRP**. Kurtosis > 3 (EQQQ.DE 65.3, IHYG.L 56.5) y asimetría negativa en 8/9 ETFs invalidan la normalidad de Markowitz puro. HRP no requiere invertir Σ ; agrupa activos por clustering jerárquico y reparte riesgo por bisección recursiva.

Generamos tres candidatas por perfil (principal, alternativa, baseline) para comparar cuantitativamente en lugar de adoptar la principal por convención.

6 Manejo del vol cap: blending con cash

Cuando la cartera optimizada w^* tiene $\sigma_p > \max_vol$, se **mezcla linealmente con cash** (sintético, $vol=0$, $retorno=rf=2\%$):

```
w_final = α · w*
cash_weight = 1 - α
α = min(1, max_vol / σ_p)
σ_final = α · σ_p ≤ max_vol
μ_final = α · μ_p + (1 - α) · rf
```

Por qué cash y no reescalado puro

Reescalar pesos por $\alpha < 1$ y **renormalizar** a suma 1 ($w / \sum w$) **no cambia la volatilidad**: es el mismo punto del simplex. La única forma matemáticamente consistente de reducir la varianza manteniendo UCITS es **añadir un activo libre de riesgo**. En la práctica, esto se traduce en una columna CASH en weights.parquet cuando aplica. Para los inputs históricos de M2, esta restricción no ha sido vinculante.

7 Resultados: 9 carteras candidatas

Métricas **ex-ante** calculadas con μ y Σ del M2 (in-sample). La validación out-of-sample se hará en M4.

Conservador (cap 8 %):

Candidata	Optimizador	Ret %	Vol %	Sharpe	HHI	Cash %
Principal	min_variance	1.24	4.35	-0.176	0.86	0
Alternativa	max_sharpe ■	1.24	4.35	-0.176	0.86	0
Baseline	equal_weight	1.41	4.68	-0.127	0.33	0

■ $\max(\mu) \approx 1.72\% < rf = 2\%$; $\max_sharpe$ cae a $\min_volatility$ con warning (§12, nota 1).

Moderado (cap 15 %):

Candidata	Optimizador	Ret %	Vol %	Sharpe	HHI	Cash %
Principal	max_sharpe	8.30	14.03	0.449	0.52	0
Alternativa	hrp	4.30	7.12	0.324	0.67	0
Baseline	equal_weight	6.55	10.55	0.432	0.33	0

Agresivo (cap 25 %):

Candidata	Optimizador	Ret %	Vol %	Sharpe	HHI	Cash %
Principal	hrp	15.14	15.22	0.863	0.41	0
Alternativa	max_sharpe	17.06	16.74	0.899	0.50	0
Baseline	equal_weight	16.09	16.12	0.874	0.33	0

8 Selección final por perfil

Criterio: **argmax(Sharpe) sujeto a $\sigma \leq \max_vol \times 1.01$** .

Perfil	Estrategia elegida	Ret %	Vol %	Sharpe	Cap
Conservador	equal_weight	1.41	4.68	-0.127	✓

Perfil	Estrategia elegida	Ret %	Vol %	Sharpe	Cap
Moderado	max_sharpe	8.30	14.03	0.449	✓
Agresivo	max_sharpe	17.06	16.74	0.899	✓

Lectura crítica:

- **Conservador.** Sharpe negativo esperable y documentado: ETFs de RF EUR con CAGR < rf=2 % (entorno ZIRP 2010-2021). El equal-weight gana porque la diversificación cruda baja el Sharpe menos que la concentración en min-variance. Con datos 2022+ (subida de tipos), la situación cambiará en M4.
- **Agresivo.** Gana max_sharpe con Sharpe 0.899, *por encima del HRP designado como principal*. El criterio de selección es agnóstico al optimizador. La 'principal' es una *prior* metodológica, no un compromiso.
- **Moderado.** Sharpe 0.449 < 0.958 del benchmark 60/40 (M2). Esto es **ex-ante** con μ in-sample; el benchmark incluye toda la deriva alcista 2010-2026. Comparación justa en M4 walk-forward.

9 Validación UCITS y de volatilidad

validators.validate_portfolio() ejecuta cuatro chequeos sobre las carteras seleccionadas:

Check	Tolerancia	Conservador	Moderado	Agresivo
sum_to_one	1e-6	✓	✓	✓
long_only	-1e-6	✓	✓	✓
vol_within_cap	× 1.01	✓	✓	✓
hhi_in_range	[1/n, 1]	✓	✓	✓
STATUS		PASS	PASS	PASS

10 Tests automatizados

Suite con **16 tests** en roboadvisor/tests/, todos pasan en ~2 s.

Fichero	N tests	Cubre
test_optimizers.py	5	Smoke tests × 4 optimizadores + 1/n
test_constraints.py	7	UCITS, HHI, effective_assets, vol cap
test_validators.py	4	Validación correcta + violaciones

confest.py define fixtures sintéticas (sample_cov 3×3 PD, sample_mu, sample_tickers, sample_max_volatility). Probar la lógica sin depender de los inputs reales.

Comando:

```
cd roboadvisor && python -m pytest tests/ -v
```

11 Outputs y handover a M4

Dos artefactos consumidos por el siguiente módulo:

weights.parquet (multi-índice profile × ticker):

```
profile    ticker    weight
```

agresivo	CSPX.L	0.5234
	EQQQ.DE	0.4766
	EXI2.DE	0.0000
conservador	EUNH.DE	0.3333
	IBCI.AS	0.3333
	IEAG.AS	0.3333
moderado	EXSA.DE	0.6106
	IHYG.L	0.0000
	INFR.AS	0.3894

portfolios_summary.json:

Diccionario {profile: {optimizer, selection, weights, cash_weight, expected_*, herfindahl_index, n_effective_assets, max_volatility, validation}}. Pensado para auditoría legible y trazabilidad de la decisión (qué algoritmo fue elegido y por qué).

12 Notas metodológicas y limitaciones

- 1. Max-Sharpe en conservador.** El algoritmo de Markowitz exige $\max(\mu) > r_f$. Si no se cumple (caso ZIRP), MaxSharpeOptimizer cae automáticamente a min_volatility y emite warning. Decisión conservadora y registrada.
- 2. In-sample bias.** Las métricas de §7 usan μ y Σ históricos sobre los *mismos datos* usados para optimizar. Son cotas optimistas. La validación honesta es M4 (walk-forward, out-of-sample).
- 3. HHI sólo sobre activos riesgo.** El índice excluye la columna CASH para que sea comparable entre carteras con y sin cash blending.
- 4. Tolerancias.** $1e-6$ para sumas y long-only; +1 % slack sobre vol cap (errores de redondeo de PyPortfolioOpt). Centralizadas en validators.py.
- 5. Reproducibilidad.** El notebook puede regenerarse desde código vía notebooks/_build_notebook.py, evitando conflictos de merge en JSON.

13 Nota metodológica: cartera ex-ante vs cartera OOS-clean

Al arrancar la implementación del **M4 (backtesting walk-forward)** se identificó una sutileza temporal en el uso de los pesos producidos por este módulo. Esta sección la documenta para que la separación de roles entre M3 y M4 quede explícita en la memoria.

El problema:

Los pesos persistidos en outputs/m3/weights.parquet se han calculado con μ y Σ estimados sobre **todo el histórico disponible** (2010-09 → 2026-05). Es decir, los parámetros del optimizador ya 'han visto' el régimen 2020-2026. Si esos pesos se aplicaran sin más sobre una ventana OOS 2020-2026, el experimento contendría **look-ahead bias en la fase de estimación de parámetros**, incumpliendo el principio walk-forward que la Fase 0 del TFM declara como requisito contractual.

La solución: dos carteras con roles disjuntos

En lugar de regenerar weights.parquet y perder la cartera 'producción' que el roboadvisor recomendaría hoy, M3 y M4 coexisten con responsabilidades claramente separadas:

Artefacto	μ y Σ	Pregunta que responde
Ex-ante (M3) outputs/m3/weights.parquet	Histórico completo 2010-09 → 2026-05	¿Qué cartera recomienda el roboadvisor hoy?

Artefacto	μ y Σ	Pregunta que responde
OOS-clean (M4) outputs/m4/weights_oos_clean.parquet	Sólo 2010-09 → 2019-12-31	¿Qué se habría recomendado en 2019 y cómo se comportó?

Ambas son **legítimas** y **necesarias**:

- La **ex-ante** es la cartera de producción: el output que un cliente real recibiría si abriera la cuenta hoy, porque incorpora toda la información disponible.
- La **OOS-clean** es la cartera de **validación científica**. Reusa los mismos optimizadores de `src/m3_portfolio/` (mismas restricciones UCITS, mismos vol caps) pero con μ y Σ recalculados sobre la ventana de entrenamiento. Sus pesos se aplican al periodo OOS para producir métricas defendibles.

Optimizador agnóstico al tiempo

Las API públicas del paquete `m3_portfolio` no cambian: los optimizadores no saben de fechas, sólo de matrices (μ , Σ). Quién decide qué ventana temporal usar es responsabilidad del **orquestador** (este notebook para M3, el notebook de backtesting para M4), no del optimizador. Esta separación permite reusar literalmente el mismo `MinVarianceOptimizer`, `MaxSharpeOptimizer` y `HRPOptimizer` en ambos contextos sin riesgo de contaminación temporal.

A Apéndice — Inventario de ficheros

Paquete (`m3_portfolio`):

- `portfolio.py` — dataclass `Portfolio`
- `base.py` — Protocol `PortfolioOptimizer`
- `constraints.py` — UCITS + vol cap + HHI
- `validators.py` — validación post-hoc
- `_build.py` — ensamblador compartido
- `min_variance.py` — `MinVarianceOptimizer`
- `max_sharpe.py` — `MaxSharpeOptimizer` (con fallback)
- `hrp.py` — `HRPOptimizer`
- `equal_weight.py` — `EqualWeightOptimizer`
- `__init__.py` — API pública

Tests (`tests/`):

- `conftest.py`
- `test_optimizers.py`
- `test_constraints.py`
- `test_validators.py`

Notebook (`notebooks/`):

- `M3_optimizacion_carteras.ipynb` — orquestador
- `_build_notebook.py` — regenerador

Outputs (`data/m3_outputs/`):

- `weights.parquet`
- `portfolios_summary.json`
- `Memoria_M3.md`

- Memoria_M3.pdf